DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COURSE: CS302 - DESIGN AND ANALYSIS OF DEEP NEURAL NETWORKS
LECTURE 02: GRADIENT DESCENT OPTIMIZATION AND BACKPROPAGATION DERIVATIONS
INSTRUCTOR: PROF. SUNIL SHARMA / DR. ARUN KUMAR

---
1. INTRODUCTION TO NUMERICAL OPTIMIZATION

In deep learning models, optimization is the process of adjusting network parameters (weights and biases) to minimize a predefined loss function L over a dataset.

For a supervised learning setup:
- Input vector: X
- Ground truth target: Y
- Predicted output: Y_hat = f(X; W, B) where W represents weight matrices and B represents bias vectors.

Common Loss Functions:
1. Mean Squared Error (MSE) for Regression:
   L = (1 / (2 * N)) * Sum((Y_i - Y_hat_i)^2)
2. Categorical Cross-Entropy for Classification:
   L = - Sum(Y_i * log(Y_hat_i))

---
2. GRADIENT DESCENT VARIANTS

A. Batch Gradient Descent
Calculates gradients of the loss function with respect to the parameters for the entire dataset in every iteration.
- Parameter Update Formula:
  W = W - eta * Grad_W(L)
- Pros: Guaranteed convergence to a local minimum for convex functions.
- Cons: Extremely slow and memory-intensive for large datasets that cannot fit in GPU VRAM.

B. Stochastic Gradient Descent (SGD)
Updates parameters using only a single training sample at a time.
- Pros: Fast execution speed per step, capable of escaping local minima due to noisy gradient updates.
- Cons: High variance in updates causes the loss trajectory to fluctuate wildly.

C. Mini-Batch Gradient Descent
Strikes a balance by computing gradients over small batches of size B (typically 32, 64, 128, or 256). This is the standard approach used in practice.

---
3. ADVANCED ADAPTIVE OPTIMIZERS
---

1. Momentum:
Incorporate a velocity vector V to smooth out gradient updates and accelerate traversal across flat surfaces.
- V_t = beta * V_{t-1} + (1 - beta) * Grad(L)

- W_{t+1} = W_t - eta * V_t

2. Adam Optimizer (Adaptive Moment Estimation):
Combines principles from Momentum (first moment) and RMSProp (second raw moment).
- First Moment Estimation: m_t = beta1 * m_{t-1} + (1 - beta1) * g_t
- Second Moment Estimation: v_t = beta2 * v_{t-1} + (1 - beta2) * (g_t)^2
- Bias Corrections:
  m_hat_t = m_t / (1 - (beta1)^t)
  v_hat_t = v_t / (1 - (beta2)^t)
- Final Update Rule:
  W_{t+1} = W_t - (eta / (sqrt(v_hat_t) + epsilon)) * m_hat_t

Default Hyperparameter Standards:
- Learning Rate (eta) = 0.001
- beta1 = 0.9
- beta2 = 0.999
- epsilon = 1e-8

---
4. THE BACKPROPAGATION ALGORITHM (MULTIVARIABLE CHAIN RULE)

Backpropagation computes the partial derivatives of the loss L with respect to every weight and bias parameter in a multi-layer computational graph using backwards sweep chain rule propagation.

For a 3-layer neural network:
- Layer 1 Linear Combination: Z_1 = W_1 * X + B_1
- Layer 1 Activation: A_1 = ReLU(Z_1)
- Layer 2 Linear Combination: Z_2 = W_2 * A_1 + B_2
- Layer 2 Activation: A_2 = Softmax(Z_2)

Gradient Computation Step-by-Step:
1. Error term at Output Layer 2:
   dZ_2 = A_2 - Y
2. Gradient w.r.t Layer 2 Weights:
   dW_2 = (1 / N) * (dZ_2 * A_1^T)
3. Gradient w.r.t Layer 2 Biases:
   dB_2 = (1 / N) * Sum_across_columns(dZ_2)
4. Backpropagate error to Hidden Layer 1:
   dA_1 = W_2^T * dZ_2
   dZ_1 = dA_1 * ReLU_derivative(Z_1)
5. Gradient w.r.t Layer 1 Weights:
   dW_1 = (1 / N) * (dZ_1 * X^T)
6. Gradient w.r.t Layer 1 Biases:
   dB_1 = (1 / N) * Sum_across_columns(dZ_1)